# Hashing

COMP 182 • Lecture 12

## Keys, collisions, and efficient lookup

Open addressing • Separate chaining • Rehashing

# Learning objectives

Compute hash codes and compress them into valid array indices.

Trace linear probing, quadratic probing, and double hashing.

Explain chaining, load factor, deletion, and rehashing.

Outline MyHashMap and MyHashSet implementations.

# Why hashing?

| Structure | Lookup | Insert / delete |
|---|---|---|
| Unsorted array | O(n) | Append O(1)* / O(n) |
| Sorted array | O(log2(n)) | O(n) / O(n) |
| Balanced search tree | O(log2(n)) | O(log2(n)) |
| Hash table | Expected O(1) | Expected amortized O(1) |

*Append assumes spare capacity. Hashing needs good distribution and load control.

# A small stored set in a large universe

U is the universe of possible keys. D is the subset currently stored.

Usually n = |D| is far smaller than |U|.

Direct addressing reserves one slot for every possible key.

Hashing uses m buckets or slots, typically proportional to n.

# A map and its implementation

A map stores one value for each distinct key.

A set stores distinct keys and supports membership tests.

A hash table is one possible implementation of either ADT.

A hash function deterministically selects a home bucket or slot.

Java: JavaHashMapDemo.java

# Map operations and replacement

| Operation | Effect |
|---|---|
| put(k, v) | Add k, or replace the value for an equal key |
| get(k) | Return the value associated with k |
| containsKey(k) | Report whether the key is present |
| remove(k) | Remove the mapping for k |

Java HashMap permits null values. get(k) == null alone cannot prove absence.

Java: JavaHashMapDemo.java

# From a key to a table index

Step 1: compute an integer hash code from the key.

Step 2: compress the code into the interval 0 through m - 1.

Example: the last four digits 1234 give 1234 mod 10 = 4.

Collisions require key comparisons and a resolution strategy.

Java: HashCodeExamples.java

# Modulo hashing and the first collision

| Key | Home index: key mod 10 | Result |
|---|---|---|
| 519 | 9 | Store in slot 9 |
| 214 | 4 | Store in slot 4 |
| 1001 | 1 | Store in slot 1 |
| 3744 | 4 | Collision with 214 |

Different keys can share a home index. A collision does not mean the keys are equal.

Java: HashCodeExamples.java

# Java hashCode and equals

Equal objects must return equal hash codes.

Unequal objects may still return the same hash code.

Override hashCode when defining value-based equals.

Keep equality-relevant key fields stable while stored.

Java: PersonKey.java

Java: PersonKeyDemo.java

# Hash codes for integral values

| Type | Typical Java computation |
| --- | --- |
| byte, short, char | Use the numeric value as an int |
| int | Integer.hashCode(key) returns key |
| long | Long.hashCode(key) combines both halves |

For long: (int) (key ^ (key >>> 32))

Java: HashCodeExamples.java

# Hash codes for floating-point values

```
int a = Float.hashCode(value);

// Equivalent double hashing steps
long bits = Double.doubleToLongBits(key);
int b = (int) (bits ^ (bits >>> 32));
```

Hash the bit representation rather than casting to an integer.

Wrapper hash methods follow wrapper equality rules.

Java: HashCodeExamples.java

# String hashing includes character position

```
int h = 0;
for (int i = 0; i < s.length(); i++) {
  h = 31 * h + s.charAt(i);
}
```

A character sum gives the same code for "ab" and "ba".

Polynomial hashing accounts for position. Collisions remain possible.

Java: HashCodeExamples.java

# Hashing an object with several fields

Choose the fields that determine equality.

Combine their hash codes so each can affect the result.

Exclude fields that vary independently of equality.

Balance computation cost against collision frequency.

Java: PersonKey.java

Java: PersonKeyDemo.java

# Compression produces a valid index

```
// Any positive table capacity m
int index = Math.floorMod(hashCode, m);

// Only when m is a power of two
int index2 = mixedHash & (m - 1);
```

Java % can return a negative result for a negative hash code.

Bit masking needs a power-of-two capacity and suitable bit mixing.

Java: HashCodeExamples.java

# Two families of collision resolution

| Method | Where colliding entries go |
| --- | --- |
| Open addressing | Other slots in the same array, using a probe sequence |
| Separate chaining | A collection of entries attached to the home bucket |

Resizing changes capacity. Reinsert entries using the new indexing rule.

Java: ProbeTable.java

Java: MyHashMap.java

# Linear probing

Probe i uses (h(key) + i) mod m, starting with i = 0.

Inspect consecutive slots and wrap from m - 1 to 0.

Lookup follows the same sequence used by insertion.

Bound a search by m probes if no never-used slot appears.

Java: ProbeTable.java

# Linear probing: a complete insertion trace

| Key | Home | Slots inspected | Stored at |
|---|---|---|---|
| 38 | 8 | 8 | 8 |
| 19 | 9 | 9 | 9 |
| 8 | 8 | 8, 9, 0 | 0 |
| 79 | 9 | 9, 0, 1 | 1 |
| 10 | 0 | 0, 1, 2 | 2 |

Capacity 10. Final occupied slots: 0:8, 1:79, 2:10, 8:38, 9:19.

Java: LinearProbingDemo.java

# Primary clustering

The occupied run spans slots 8, 9, 0, 1, 2.

Inserting 18 probes 8, 9, 0, 1, 2, 3 and stores at 3.

Keys whose home slots fall in the run extend the same cluster.

Long clusters increase both lookup and insertion work.

Java: LinearProbingDemo.java

# Open-addressed deletion preserves the trail

| Slot state | Lookup action | Insertion action |
|---|---|---|
| LIVE | Compare key, then continue if unequal | Update equal key |
| DELETED | Continue probing | Remember as a reusable position |
| EMPTY | Stop: key is absent | Insert, using an earlier tombstone if any |

Example: delete 38 from slot 8. Lookup for 8 must still reach slot 0.

Java: LinearProbingMap.java

Java: TombstoneMapDemo.java

# Bounded lookup with tombstones

```
find(key):
  for i = 0 to m - 1:
    j = probe(key, i)
    if state[j] == EMPTY: return MISSING
    if state[j] == LIVE and equals(keys[j], key):
      return values[j]
  return MISSING
```

DELETED slots keep the search going.

Rebuild periodically when deleted slots accumulate.

Java: LinearProbingMap.java

Java: TombstoneMapDemo.java

# Linear probing practice: six keys

Insert in order: 44, 4, 16, 28, 21, 26.

Use capacity m = 11 and home index key mod 11.

Use linear probing: (key mod 11 + i) mod 11.

Start i = 0; record every probe and the final table.

Java: ProbingPractice.java

# Practice answer: linear probing

| Key | Home | Slots inspected | Stored at |
|---|---|---|---|
| 44 | 0 | 0 | 0 |
| 4 | 4 | 4 | 4 |
| 16 | 5 | 5 | 5 |
| 28 | 6 | 6 | 6 |
| 21 | 10 | 10 | 10 |
| 26 | 4 | 4, 5, 6, 7 | 7 |

Occupied slots: 0:44, 4:4, 5:16, 6:28, 7:26, 10:21.

Java: ProbingPractice.java

# Quadratic probing

Probe i uses (h(key) + $i^2$) mod m.

Offsets begin 0, 1, 4, 9, 16, 25, ...

This avoids the contiguous runs created by linear probing.

Keys with the same home slot still share a sequence.

Java: ProbeTable.java

Java: QuadraticProbingDemo.java

# Quadratic probing with capacity 10

| Key | Home | Slots inspected | Stored at |
|---|---|---|---|
| 89 | 9 | 9 | 9 |
| 18 | 8 | 8 | 8 |
| 49 | 9 | 9, 0 | 0 |
| 58 | 8 | 8, 9, 2 | 2 |
| 79 | 9 | 9, 0, 3 | 3 |

The offset is $i^2$ from the original home slot on every probe.

Java: QuadraticProbingDemo.java

# Quadratic probing can miss empty slots

| Key | Probe path until placement | Stored at |
|---|---|---|
| 76 | 6 | 6 |
| 40 | 5 | 5 |
| 48 | 6, 0 | 0 |
| 5 | 5, 6, 2 | 2 |
| 55 | 6, 0, 3 | 3 |
| 47 | 5, 6, 2, 0, 0, 2, 6 | Fails |

Capacity 7. Slots 1 and 4 remain empty, but 47 cannot reach them.

Java: QuadraticProbingDemo.java

# Quadratic probing practice: six keys

Insert in order: 44, 4, 16, 28, 21, 26.

Use capacity m = 11 and home index key mod 11.

Use quadratic probing: (key mod 11 + $i^2$) mod 11.

Start i = 0; record every probe and the final table.

# Practice answer: quadratic probing

| Key | Home | Slots inspected | Stored at |
|---|---|---|---|
| 44 | 0 | 0 | 0 |
| 4 | 4 | 4 | 4 |
| 16 | 5 | 5 | 5 |
| 28 | 6 | 6 | 6 |
| 21 | 10 | 10 | 10 |
| 26 | 4 | 4, 5, 8 | 8 |

Occupied slots: 0:44, 4:4, 5:16, 6:28, 8:26, 10:21.

Java: QuadraticProbingPractice.java  Java: ProbeTable.java

# Double hashing

Probe i uses (h1(key) + i × h2(key)) mod m.

h1 gives the home slot. h2 gives a key-dependent step.

A nonzero step is necessary. Full coverage also needs gcd(step, m) = 1.

Different keys can still share both hash results.

# A double-hashing configuration that fails

| Key | Step | Slots inspected | Result |
|---|---|---|---|
| 13 | 2 | 3 | Stored at 3 |
| 28 | 3 | 8 | Stored at 8 |
| 33 | 4 | 3, 7 | Stored at 7 |
| 147 | 6 | 7, 3, 9 | Stored at 9 |
| 43 | 5 | 3, 8, 3, 8, ... | Fails |

m = 10. h1(k) = k % 10. h2(k) = 1 + ((k / 10) % 9), using integer division.

Java: DoubleHashingDemo.java

# A full-coverage double-hashing scheme

Choose a prime capacity p and a smaller prime q.

h1(key) = floorMod(key, p).

h2(key) = q - floorMod(key, q).

Every step lies between 1 and q, so it is coprime to p.

Java: FullCoverageDoubleHashingDemo.java

Java: ProbeTable.java

# Double Hashing practice: six keys

Insert in order: 44, 4, 16, 28, 21, 26.

Use capacity m = 11 and home index key mod 11.

Use h2: 7 – (key mod 7).

Start i = 0; record every probe and the final table.

# Practice answer: double hashing

| Key | Home | Slots inspected | Stored at |
|---|---|---|---|
| 44 | 0 | 0 | 0 |
| 4 | 4 | 4 | 4 |
| 16 | 5 | 5 | 5 |
| 28 | 6 | 6 | 6 |
| 21 | 10 | 10 | 10 |
| 26 | 4 | 4, 6, 8 | 8 |

Occupied slots: 0:44, 4:4, 5:16, 6:28, 8:26, 10:21.

Java: DoubleHashingPractice.java

# Separate chaining

Each array bucket holds all entries that hash to that index.

A linked list is a common bucket representation.

Lookup compares keys within the selected bucket.

Deletion unlinks an entry. Tombstones are unnecessary.

Java: MyHashMap.java

Java: ChainingDemo.java

# Chaining: five insertions

| Inserted key | Bucket: key mod 10 | Bucket after head insertion |
|---|---|---|
| 10 | 0 | [10] |
| 22 | 2 | [22] |
| 86 | 6 | [86] |
| 12 | 2 | [12, 22] |
| 42 | 2 | [42, 12, 22] |

Final nonempty buckets: 0:[10], 2:[42, 12, 22], 6:[86].

Java: ChainingDemo.java

# Load factor depends on the representation

| Quantity | Definition or example |
| --- | --- |
| Load factor alpha | n / m, entries divided by capacity |
| Open addressing | n <= m. Keep alpha below 1 for usable slack. |
| Chaining, 5 entries / 10 buckets | alpha = 0.5 |
| Chaining, 21 entries / 10 buckets | alpha = 2.1 is valid |

A single universal threshold such as 2/3 does not apply to every design.

Java: ChainingDemo.java

Java: HashAnalysisExamples.java

# Chaining with 21 entries

| Bucket | Chain, head first |
|---|---|
| 0 / 1 / 2 | [10] / [71, 31] / [42, 12, 22, 2] |
| 3 / 4 / 5 | [63, 73] / empty / [75, 5, 65, 95] |
| 6 / 7 | [86] / [27, 47] |
| 8 / 9 | [88, 18, 38, 98] / [99] |

21 entries in 10 buckets give alpha = 2.1.

Java: ChainingDemo.java

# Rehashing changes the table layout

Allocate a larger table when the chosen threshold is reached.

Recompute each live entry’s index for the new capacity.

Reinsert using the collision policy of the new table.

Discard tombstones and update capacity, size, and threshold.

Java: RehashingDemo.java

Java: TombstoneMapDemo.java

# Linear probing: load and expected probes

| alpha | Successful search | Unsuccessful search |
|---|---|---|
| 0.50 | 1.50 | 2.50 |
| 0.75 | 2.50 | 8.50 |
| 0.90 | 5.50 | 50.50 |

Classical estimates: $S \approx \frac{1}{2}(1 + 1/(1-\text{alpha}))$, $U \approx \frac{1}{2}(1 + 1/(1-\text{alpha})^2)$.

Java: HashAnalysisExamples.java

# Comparing collision strategies

| Strategy | Main cost or limitation |
| --- | --- |
| Linear probing | Primary clusters grow as occupancy rises |
| Quadratic probing | Secondary clustering and restricted slot coverage |
| Double hashing | Requires a step coprime to capacity |
| Separate chaining | Long chains cost comparisons and node space |

Expected constant-time access requires distribution and load assumptions.

Java: ProbeTable.java

Java: MyHashMap.java

# A weak hash can favor some buckets

Suppose a nine-digit key includes a two-digit prefix from 00 to 99.

Hash only the prefix with prefix mod 40.

Buckets 0–19 each receive three possible prefixes.

Buckets 20–39 each receive only two possible prefixes.

Java: HashAnalysisExamples.java

# Legacy supplemental mixing

```
// Historical mixing example from the source
h ^= (h >>> 20) ^ (h >>> 12);
h = h ^ (h >>> 7) ^ (h >>> 4);
int index = h & (capacity - 1);
```

>>> shifts in zeros. ^ combines bits with exclusive OR.

Mixing can move information from high bits into low bits.

Java: HashMixingExamples.java

# OpenJDK HashMap: mixing before masking

```
// OpenJDK 21 implementation excerpt
static final int hash(Object key) {
  int h;
  return (key == null) ? 0
      : (h = key.hashCode()) ^ (h >>> 16);
}

// For an allocated table of power-of-two length n
int index = (n - 1) & hash(key);
```

Implementation details can change. The Map interface does not prescribe them.

Java: HashMixingExamples.java

# Java HashMap in practice

HashMap provides key-based access without a sorted iteration order.

Its default load-factor setting is 0.75.

Sizing for the expected entry count can reduce resizing.

Tree-based maps support ordered operations.

Java: JavaHashMapDemo.java

# MyHashMap: chained put outline

```
put(key, value):
  bucket = table[index(key, capacity)]
  for entry in bucket:
    if equals(entry.key, key):
      old = entry.value
      entry.value = value
      return old
  prepend new Entry(key, value) to bucket
  size = size + 1
  if size > threshold: rehash()
  return MISSING
```

A replacement changes the value without increasing size.

Java: MyHashMap.java

Java: RehashingDemo.java

# MyHashSet can reuse a map

```
add(key):
  wasPresent = map.containsKey(key)
  map.put(key, PRESENT)
  return not wasPresent

contains(key): return map.containsKey(key)

remove(key):
  wasPresent = map.containsKey(key)
  map.remove(key)
  return wasPresent
```

PRESENT is one shared marker value. Keys remain unique.

Java: MyHashSet.java

Java: MyHashSetDemo.java

# Review questions

Why can two unequal keys have the same hash code?

Why does a deleted slot keep an open-addressed lookup going?

Why does step 5 fail with capacity 10?

Why does resizing require reinsertion?

# Further study and classroom activities

Repeat the six-key exercise with a different insertion order.

Compare successful and unsuccessful probe counts.

Delete a key, reuse its tombstone, then verify every lookup.

Implement chaining first, then add resizing and a set wrapper.

Java: ProbingPractice.java

Java: TombstoneMapDemo.java

Java: ChainingDemo.java